

Programmierung

Vorlesung, Übung und Tutorium

03 – Argumente und Kommandozeilenbefehle

Prof. Dr. Andreas Biesdorf

Tutorium: Nicolas Pfeifer

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

Agenda

1

Software Engineering vs.
Programmierung

2

Testen in Python

3

Argumente und
Kommandozeilenbefehle

4

Datei-Handling und IO

5

Algorithmen und Datenstrukturen

6

Such- und Sortierverfahren

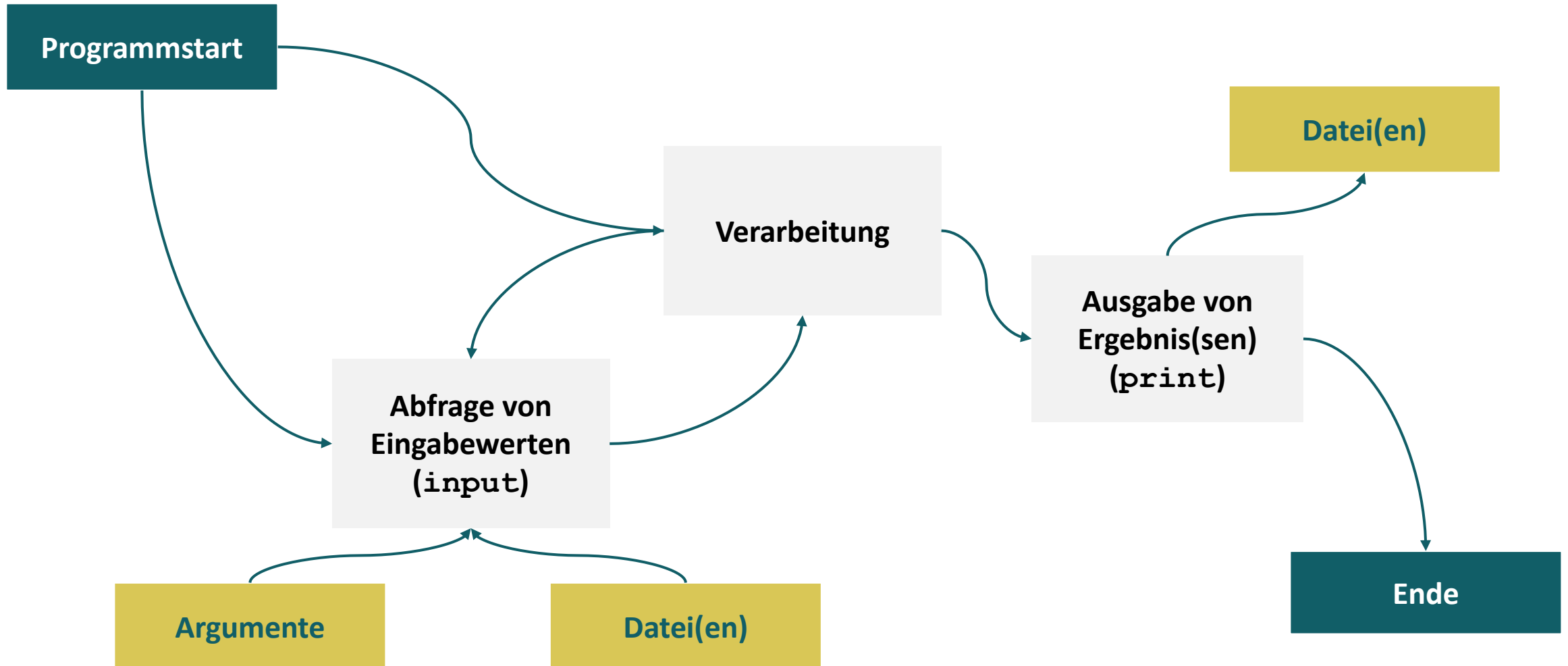


Portfolioprüfung

UNSERE PROGRAMME BISHER

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R



NACHTEILE DER EINGABE MIT `INPUT()` UND AUSGABE MIT `PRINT()`

`input()`

- Schwierigkeiten bei **großen Datenmengen** (z.B. Bilder oder Log-Files)
- **Blockierend** bei Skripten

`print()`

- Nur **einmalige Ausgabe** der Daten auf der Kommandozeile
- “**Piping**“ der Kommandozeilenausgabe in Datei möglich, aber **umständlich**

3

Argumente und
Kommandozeilenbefehle

4

Datei-Handling und IO

- Übergabe von durch Leerzeichen getrennte Argumente an den Python Interpreter

```
python test.py argument1 argument2
```

Durch Leerzeichen getrennte Strings

- Auswertung im Code mithilfe des sys-Moduls

```
import sys

if len(sys.argv) > 1:
    for arg in range(len(sys.argv)):
        print('Argument ' + str(arg) + ':', sys.argv[arg])
else:
    print('Keine Argumente angegeben')
```

- Schreiben Sie ein Programm, welches in Abhängigkeit der angegebenen Parameter auf Deutsch oder Englisch den Nutzer mit seinem Namen begrüßt oder verabschiedet:
 - ```
>> python test.py de g Andreas
```

```
>> Hallo, Andreas!
```
  - ```
>> python test.py en g Andreas
```

```
>> Hello, Andreas!
```
 - ```
>> python test.py de b Peter
```

```
>> Tschüss, Peter!
```
  - ```
>> python test.py en b Klaus
```

```
>> Bye, Klaus!
```

- **Reihenfolge** der Argumente nicht immer gleich
- Angabe von **Lang- / Kurzversionen** von Argumenten
- **Manuelles Parsing** der Eingaben erforderlich – sehr Fehleranfällig!
- Schwierig in **Integrations-Tests** einzubinden

Wunsch:

- **Systematische** Angabe von Parametern in Lang- und Kurzversionen und in beliebiger Reihenfolge

```
python test.py -l de -t g -name Andreas
```

- Parser für Kommandozeileneingaben

Zu verarbeitende
Argumente,
typischerweise
sys.argv[1:]

unterstützte lange
Varianten der
Argumente

```
getopt.getopt(args, shortopt, longopts)
```

unterstützte kurze
Varianten der
Argumente

<https://docs.python.org/3/library/getopt.html>

- Ein Flag ist ein **speziell ausgezeichnetes Argument**
- **Kurzes Flag** beginnt mit - / **langes Flag** beginnt mit --

```
python test.py -f ./dateiname.txt
```

```
python test.py --filename ./dateiname.txt
```
- Wenn Flag fehlt, wird der **Standardwert** verwendet

- **Benennt die möglichen Flags:**

Beispiel: 'hab'

```
python test.py -h -a -b
```

- Falls ein Flag ein **Argument** erfordert, ergänzen wir einen Doppelpunkt

Beispiel: 'ha:b'

```
python test.py -h -a argument1 -b
```

- **Benennt die möglichen Flags als Worte:**

Beispiel: ['help', 'simplex']

```
python test.py --help
```

- Falls ein Flag ein **Argument** erfordert, ergänzen wir ein Gleichheitszeichen

Beispiel: ['help', 'simplex', 'option=']

```
python test.py --simplex --option=argument1
```

ERGEBNIS VON GETOPTS

- `opts`: Liste von Flags mit zugeordneten Argumenten
- `args`: Argumente ohne Flags

```
import sys
import getopt

args=sys.argv[1:]
inputfile = ''
outputfile = ''

try:
    opts, args = getopt.getopt(args,"hi:o:",["infile=", "outfile="])
except getopt.GetoptError:
    print ('test.py -i <inputfile> -o <outputfile>')
    sys.exit(2)

for opt, arg in opts:
    if opt == '-h':
        print ('args.py -i <inputfile> -o <outputfile>')
        sys.exit()
    elif opt in ("-i", "--infile"):
        inputfile = arg
    elif opt in ("-o", "--outfile"):
        outputfile = arg

print ('Input file is "', inputfile)
print ('Output file is "', outputfile)
```

AKTUELLER ANSATZ: ARGPARSE

```
import argparse

def main():
    parser = argparse.ArgumentParser(description="Ein kleines Beispiel")
    parser.add_argument('zahl', type=int, help="Eine ganze Zahl eingeben")
    parser.add_argument('--verbose', action='store_true', help="Mehr Details ausgeben")
    args = parser.parse_args()

    ergebnis = args.zahl ** 2
    if args.verbose:
        print(f"Das Quadrat von {args.zahl} ist {ergebnis}.")
    else:
        print(ergebnis)

if __name__ == "__main__":
    main()
```